



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	Sheila Aqrianggita Rolanda
NPM	24282420129
Kelas	SI5C
Topik Tugas 1	Recipes
Entitas utama	
Tautan repositori	
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	Selasa, 06-10-2026

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Manajemen Resep / RESTful API Resep Makanan
Entitas utama (tunggal)	recipe (<i>atau Resep</i>)
Alamat dasar	/api/recipes
Field data	id, title, ingredients, instructions, prepTime
Field wajib saat POST	title, ingredients, instructions, prepTime
Query string untuk filter	title (<i>atau prepTime</i>)
Data awal	Array berisi objek resep default (misal: ID 1, title: "Nasi Goreng", prepTime: 15)
Modul lain (jika ada)	dotenv, express, cors

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/[sumber-daya]	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/[sumber-daya]/:id	Tidak	200	Ambil satu data
3	POST	/[sumber-daya]	Ya	201	Tambah data
4	PUT	/[sumber-daya]/:id	Ya	200	Ubah data
5	DELETE	/[sumber-daya]/:id	Ya	204	Hapus data

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.

```
(c) Microsoft Corporation. All rights reserved.
C:\Users\sheilarolanda>mkdir si5c_kuis1_sheilaarolanda
C:\Users\sheilarolanda>cd si5c_kuis1_sheilaarolanda
C:\Users\sheilarolanda\si5c_kuis1_sheilaarolanda>npm init -y
Wrote to C:\Users\sheilarolanda\si5c_kuis1_sheilaarolanda\package.json:

{
  "name": "si5c_kuis1_sheilaarolanda",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

C:\Users\sheilarolanda\si5c_kuis1_sheilaarolanda>npm install express cors dotenv
added 71 packages, and audited 72 packages in 3s
30 packages are looking for funding
```

Gambar 1. Versi Node.js dan instalasi paket

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

Pada langkah ini, dilakukan inisialisasi `package.json` dan instalasi tiga perpustakaan utama:

1. **Express (express)** : *Framework* web untuk mengelola rute API (GET, POST, PUT, DELETE), *middleware* , dan respon JSON.
2. **CORS (cors)** : *Middleware* untuk mengizinkan akses API dari domain lain agar tidak memblokir browser.
3. **dotenv (dotenv)** : Memuat variabel lingkungan dari file `.env` ke `process.env` untuk menyembunyikan data sensitif seperti `PORT` dan `API_KEY`.

Alasan Syarat Versi Node.js:

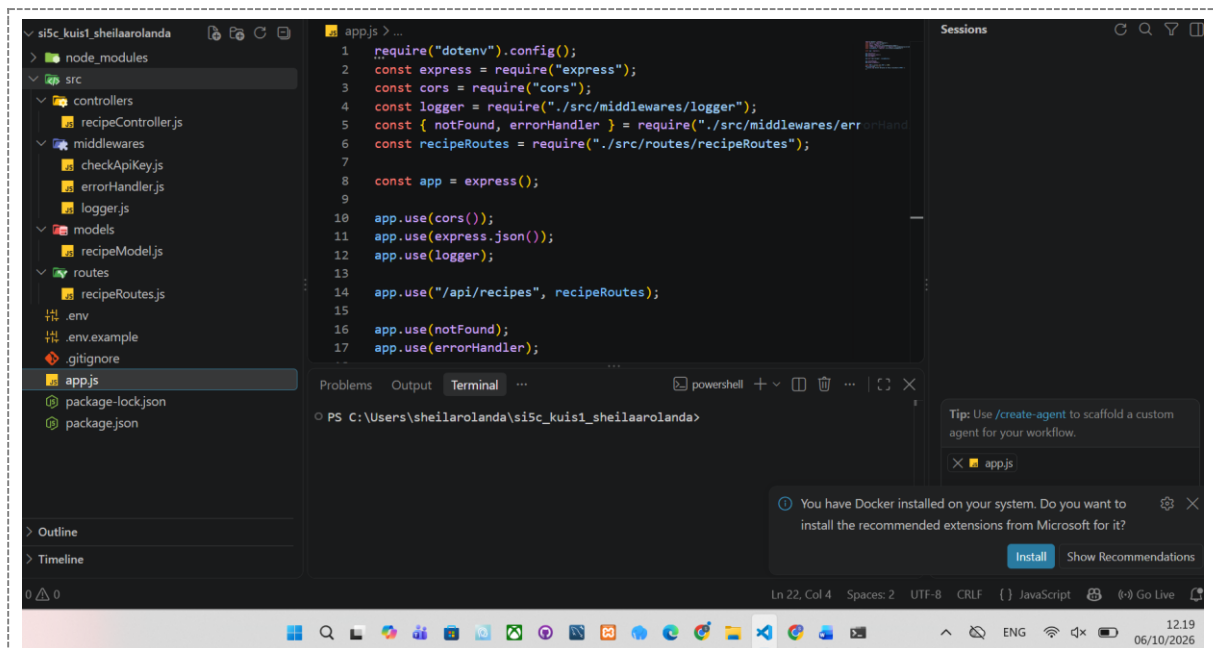
Express 5 membutuhkan Node.js versi baru (v18+) karena menggunakan fitur JavaScript modern, optimasi mesin V8, dan menangani *async/await error* secara langsung agar server berjalan stabil tanpa kendala kompatibilitas.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Gambar 2. Struktur folder project

Penjelasan:

- **Folder routes :** Berfungsi mendefinisikan seluruh URL titik akhir serta mengarahkan permintaan HTTP ke pengontrol yang sesuai.
- **Folder controllers :** Berfungsi menangani logika bisnis aplikasi dan mengontrol respons HTTP untuk setiap endpoint .
- **Folder models :** Berfungsi mengelola struktur data serta operasi manipulasi data resep secara ringkas.
- **Folder middlewares :** Berfungsi menampung fungsi perantara untuk verifikasi header, pencatatan log , dan penanganan error .

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

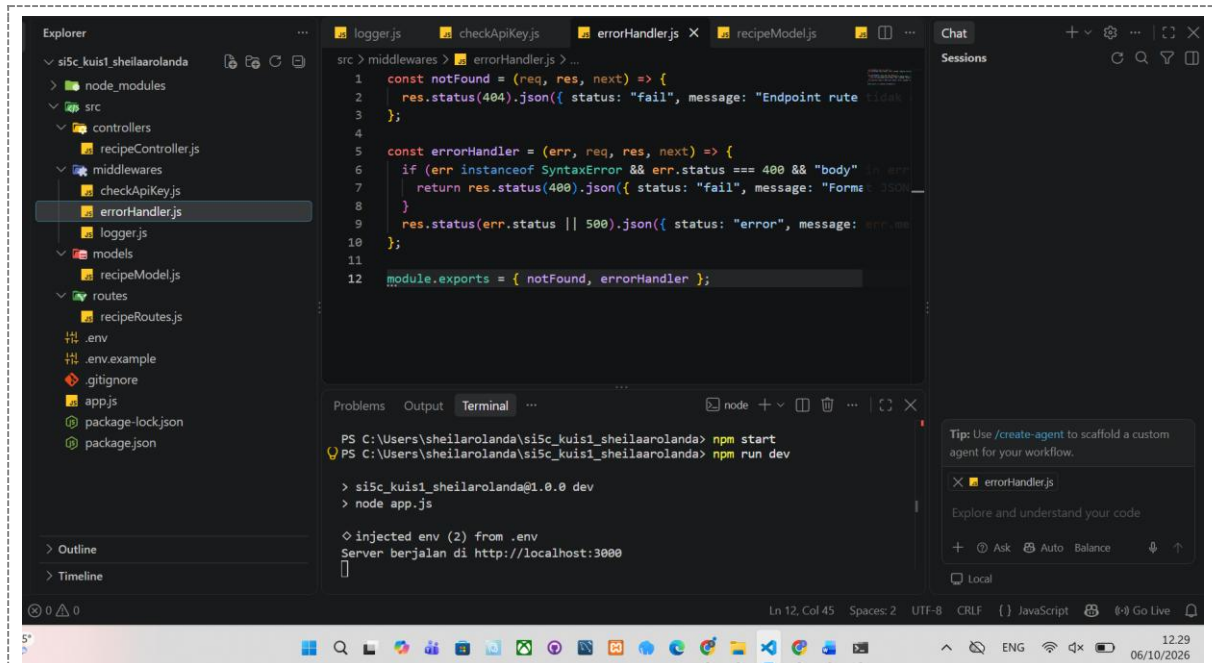
Berkas	Penjelasan Anda
models/recipeModel.js	menyimpan data array recipes dan fungsi CRUD (getAll, create, update, dll).

Berkas	Penjelasan Anda
controllers/recipeController.js	Mengolah logika permintaan HTTP dan mengirim status respons JSON.
routes/recipeRoutes.js	Mengatur konfigurasi URL ke controller dan proteksi middleware.
middlewares/logger.js	Mencatat log <i>stempel waktu</i> , metode, dan URL ke konsol
middlewares/cekApiKey.js	Memeriksa validitas header x-api-key dengan process.env.API_KEY
middlewares/errorHandler.js	berpartisipasi error rute tak ditemukan (404) dan format JSON rusak (400).
app.js	Berkas utama untuk mengaktifkan Express, middleware, rute, dan port server.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah npm start atau npm run dev dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Gambar 3. Server berhasil dijalankan

Penjelasan:

Nomor port 3000 diambil secara dinamis dari berkas .env (PORT=3000) yang dibaca melalui perpustakaan dotenv. Jika port 3000 sudah digunakan oleh aplikasi lain, server akan mengalami error EADDRINUSE dan gagal berjalan. Solusinya adalah menutup aplikasi yang menggunakan port tersebut atau mengganti nilai PORT dalam berkas .env ke nomor port lain.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

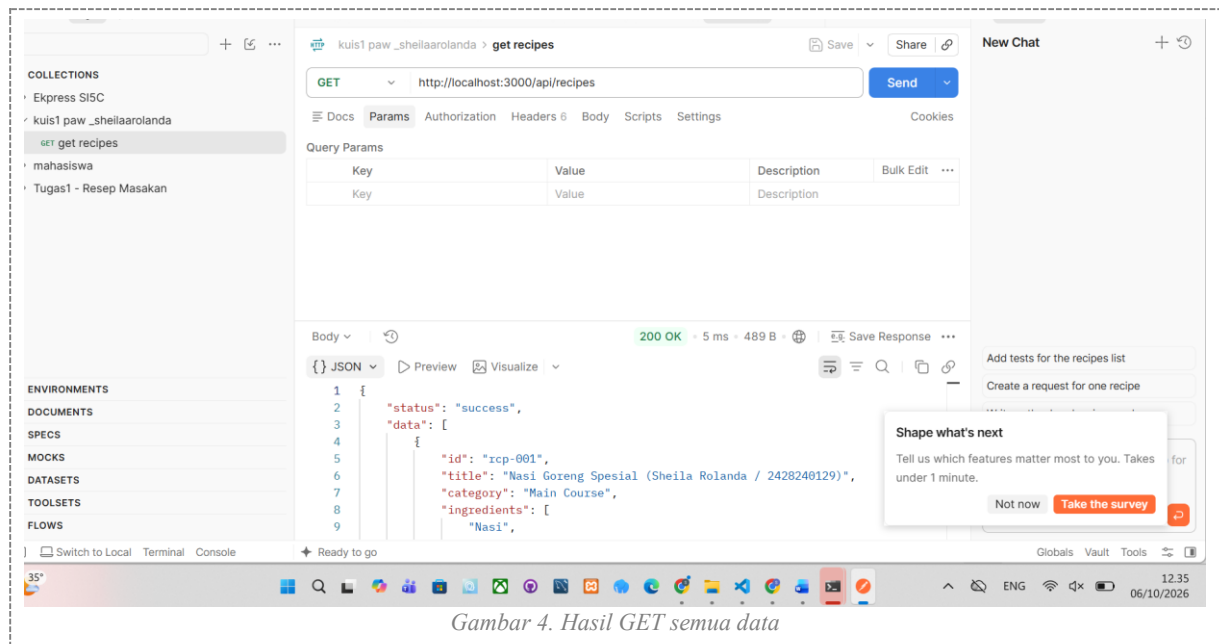
Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji". Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[sumber-daya])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /[sumber-daya]
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 4. Hasil GET semua data

Gambar 4. Hasil GET semua data

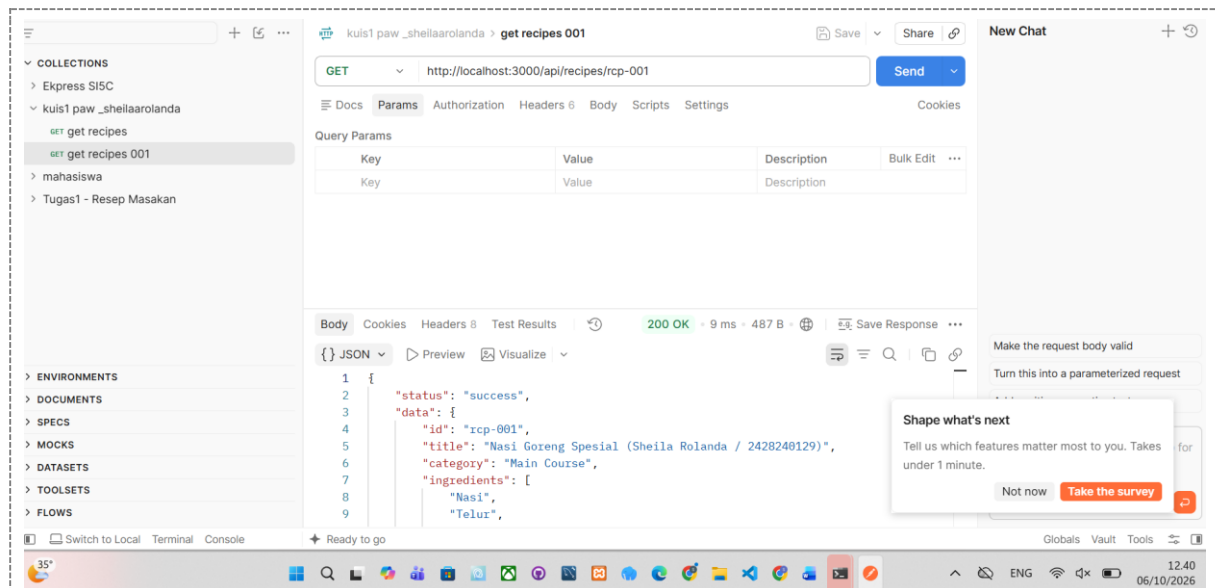
Penjelasan:

Pengujian ini bertujuan untuk mengambil seluruh resep data yang tersimpan di sistem. Permintaan dikirim menggunakan metode GET ke endpoint `/api/recipe` tanpa memerlukan header khusus maupun request body. Sistem menyelesaikan proses permintaan dan mengembalikan kode status 200 OK beserta daftar resep dalam bentuk larik JSON.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (<i>request</i>)	GET <code>/[sumber-hari]/[id yang ada]</code>
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 5. Hasil GET berdasarkan ID

Gambar 5. Hasil GET berdasarkan ID

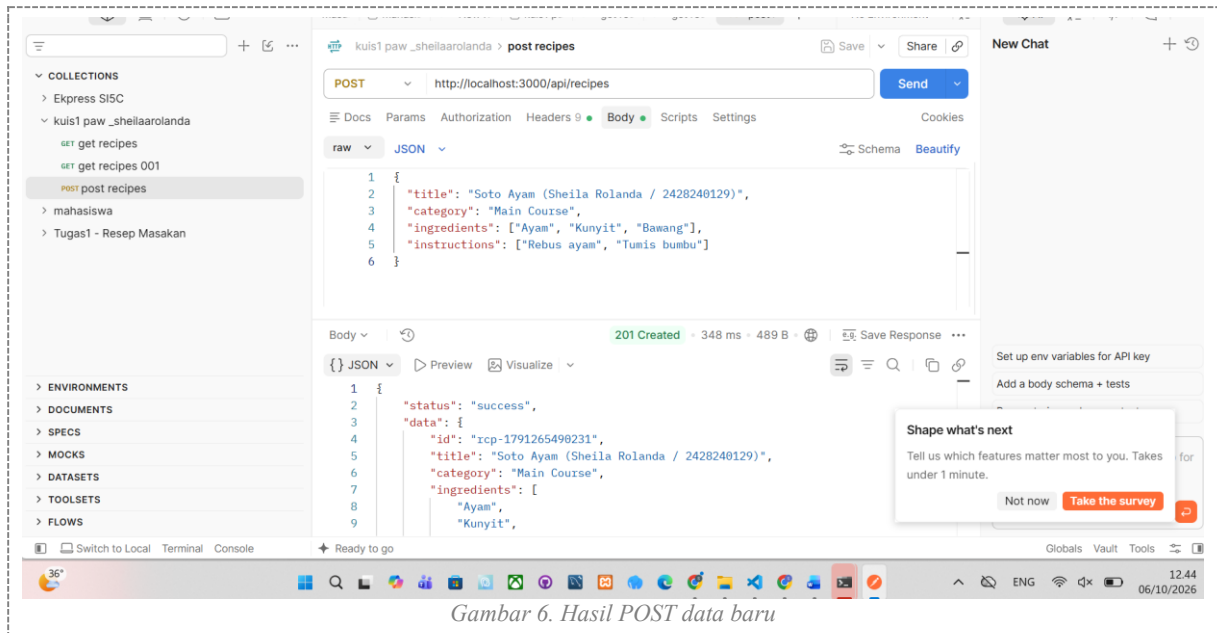
Penjelasan:

Pengujian ini dilakukan untuk mengambil detail satu data resep secara spesifik berdasarkan parameter ID (rcp-001). permintaan dikirim dengan metode GET ke endpoint `/api/recipes/rcp-001`. Server berhasil menemukan data dan mengembalikan kode status 200 OK berisi satu objek tunggal resep JSON.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header `x-api-key` dan `Content-Type`, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (<i>request</i>)	POST <code>/[sumber-hari]</code>
Header	<code>x-api-key</code> : (nilai <code>API_KEY</code> pada <code>.env</code> Anda) <code>Content-Type</code> : <code>application/json</code>
Body	<code>{ "[field wajib 1]": "...", "[field wajib 2]": "..." }</code>
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 6. Hasil POST data baru

Gambar 6. Hasil POST data baru

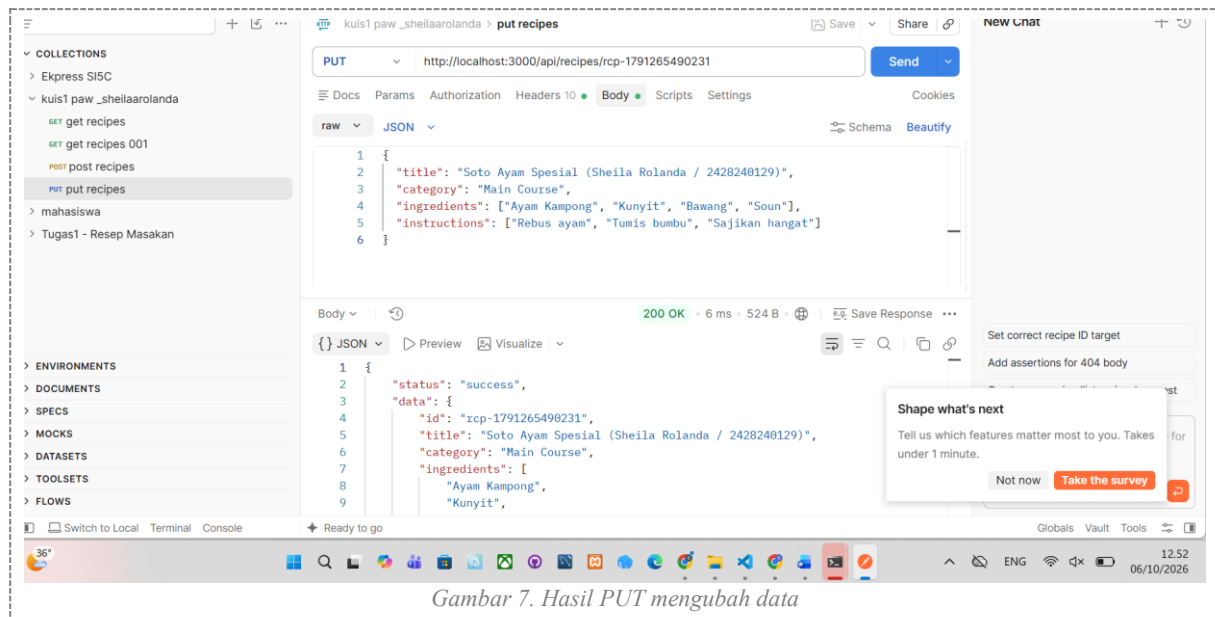
Penjelasan:

Pengujian ini bertujuan untuk menambahkan data resep baru ke dalam sistem dengan autentikasi api-key. Permintaan dikirim menggunakan metode POST ke endpoint `/api/recipes` dengan melampirkan header `x-api-key` serta `body` JSON yang memuat identitas nama dan NPM. Server berhasil memverifikasi kunci API, menyimpan data baru, dan mengembalikan respon berstatus 201 Created beserta objek resep lengkap dengan ID yang dibuat secara otomatis.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (<i>request</i>)	PUT <code>/[sumber-daya]/[id data dari T03]</code>
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "[field]": "[nilai baru]" }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 7. Hasil PUT mengubah data

Gambar 7. Hasil PUT mengubah data

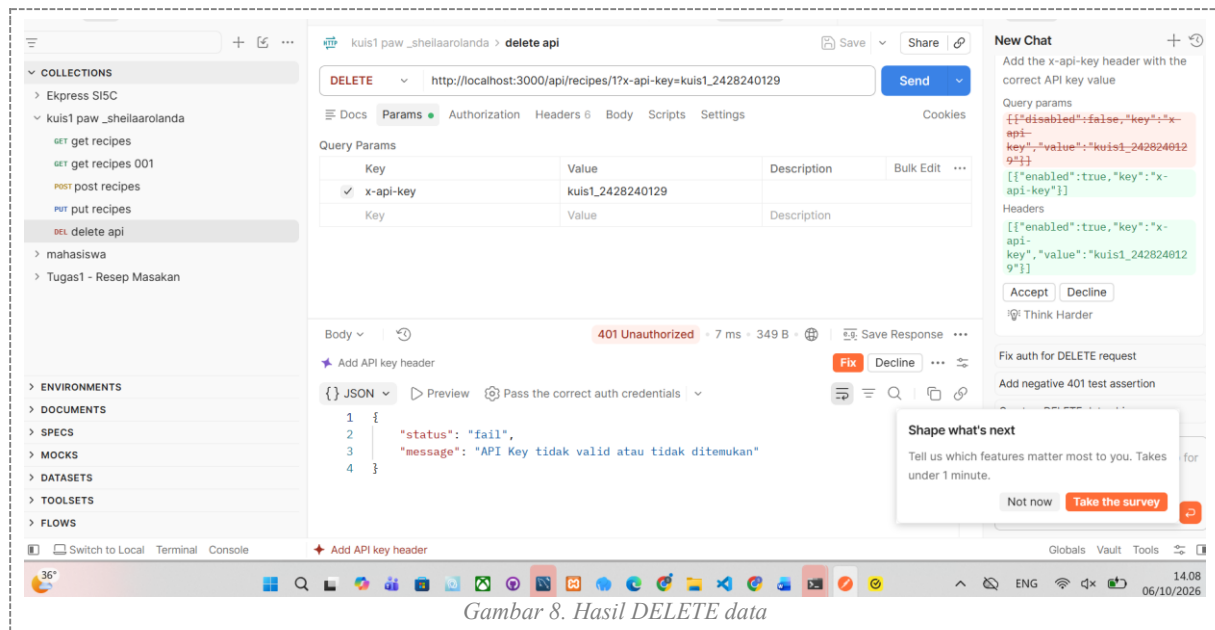
Penjelasan:

Pengujian ini bertujuan untuk memperbarui data resep yang telah ditambahkan pada T03 berdasarkan ID-nya. Permintaan dikirim dengan metode PUT ke endpoint /api/recipes/rcp-004 membawa header x-api-key serta payload JSON baru. Server berhasil memverifikasi autentikasi, mengupdate data target, dan mengembalikan respon berstatus 200 OK berisi objek resep yang telah diperbarui.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /[sumber-hari]/[id data dari T03]
Header	x-api-key: (nilai API_KEY pada .env Anda)
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 8. Hasil DELETE data

Gambar 8. Hasil DELETE data

Penjelasan:

Pengujian ini bertujuan untuk menghapus data resep berdasarkan ID yang valid. permintaan dikirim dengan metode DELETE untuk /api/recipes/rcp-1791265490231 melampirkan header x-api-key. Server berhasil memverifikasi autentikasi, menghapus data target dari sistem, dan mengembalikan respons berstatus 204 No Content tanpa isi respons

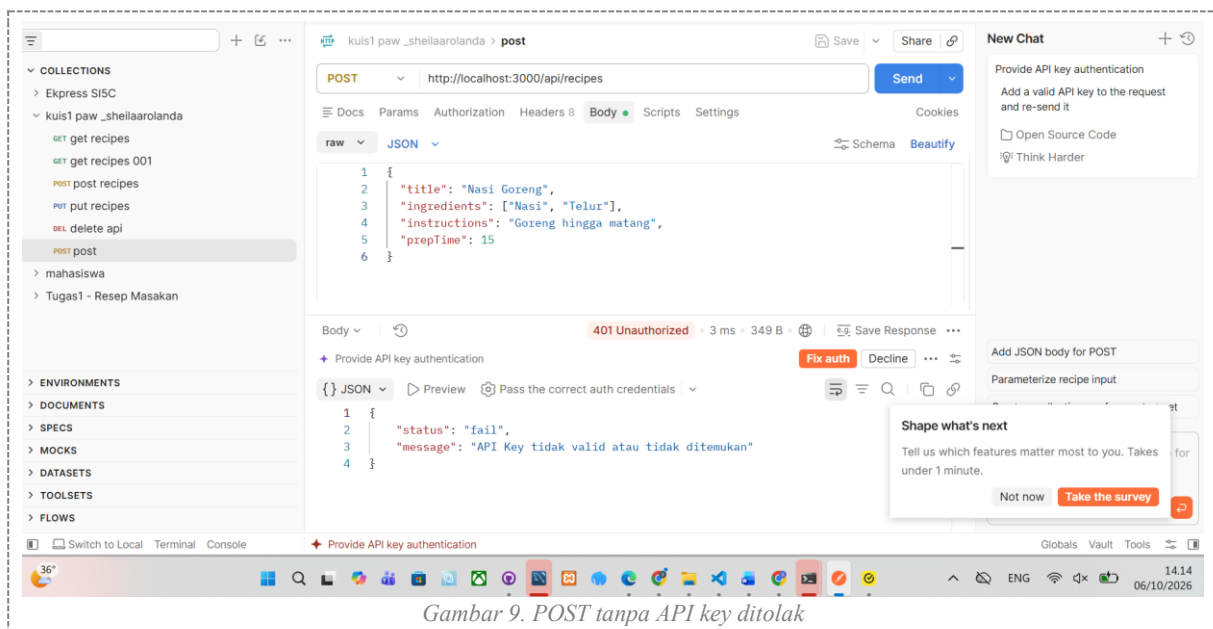
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (<i>request</i>)	POST /[sumber-harga] (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ data yang valid }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 9. POST tanpa API key ditolak

Gambar 9. POST tanpa API key ditolak

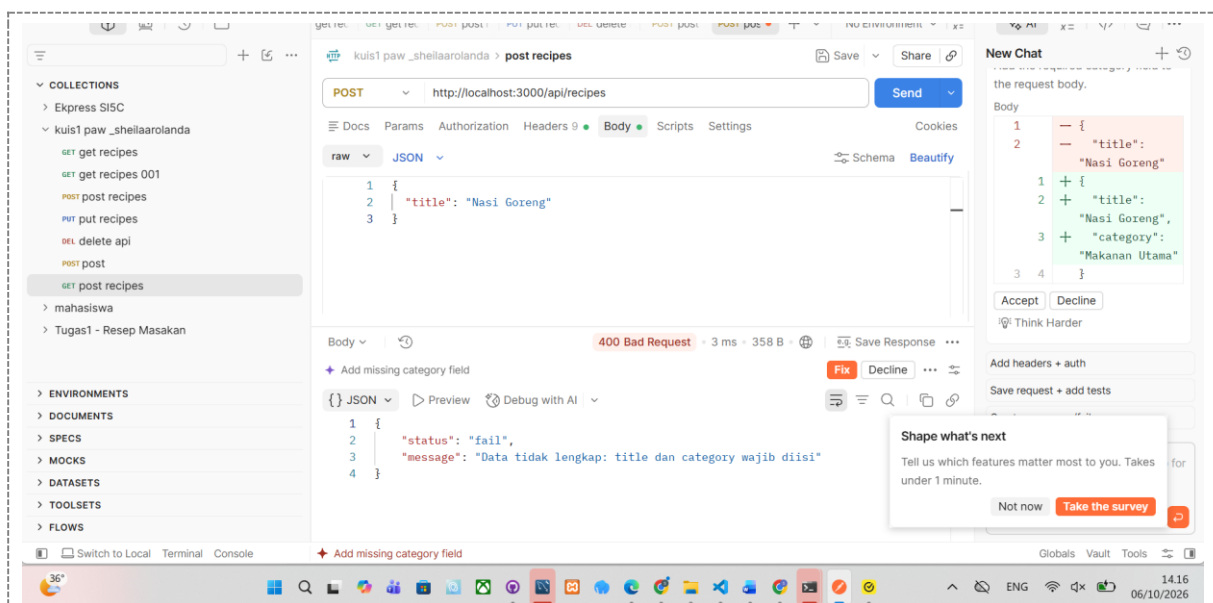
Penjelasan:

Pengujian ini menguji keamanan autentikasi saat menambahkan data tanpa melampirkan header x-api-key. permintaan POST dikirim ke /api/recipes dengan body yang valid. Middleware checkApiKey berhasil mencegah permintaan ini dan mengembalikan status 401 Unauthorized beserta pesan error JSON sehingga API Key tidak ditemukan.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (<i>request</i>)	POST /[sumber-daya]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ hanya sebagian field wajib }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 10. POST dengan body tidak lengkap

Gambar 10. POST dengan body tidak lengkap

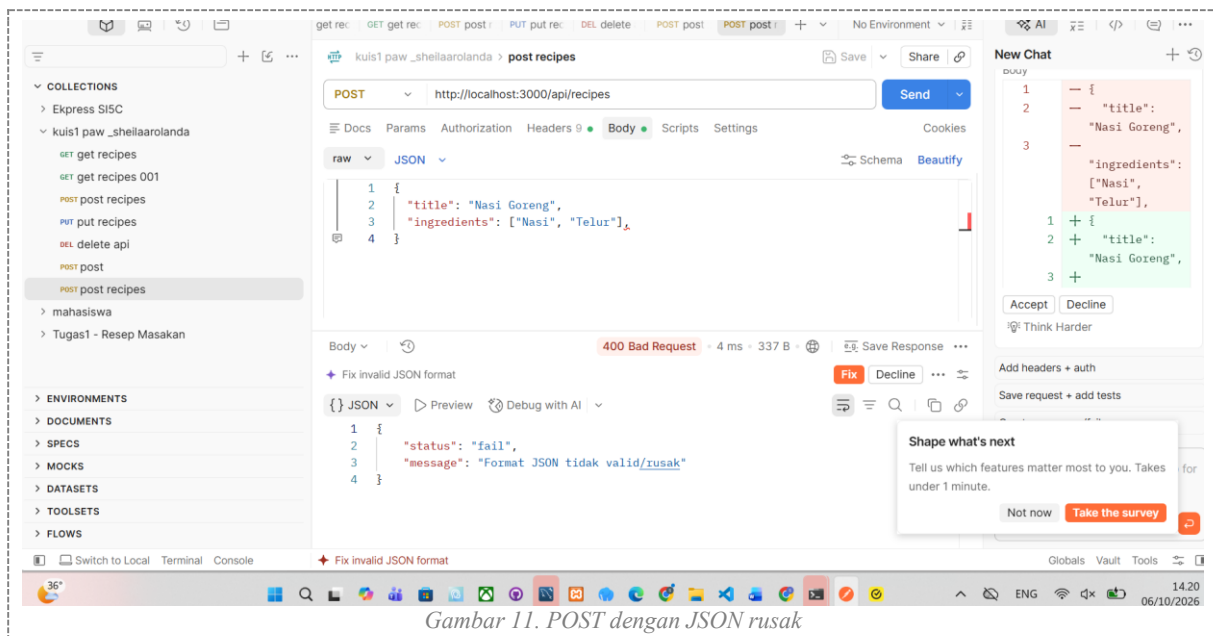
Penjelasan:

Pengujian ini bertujuan untuk memvalidasi kelengkapan data lapangan wajib saat membuat resep baru. Permintaan dikirim melalui POST ke /api/recipes membawa x-api-key, tetapi sengaja mengosongkan beberapa field wajib di *body*. Controller berhasil memvalidasi input dan mengembalikan status 400 Bad Request beserta pesan error JSON.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (<i>request</i>)	POST /[sumber-hari]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ JSON dengan koma berlebih }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 11. POST dengan JSON rusak

Gambar 11. POST dengan JSON rusak

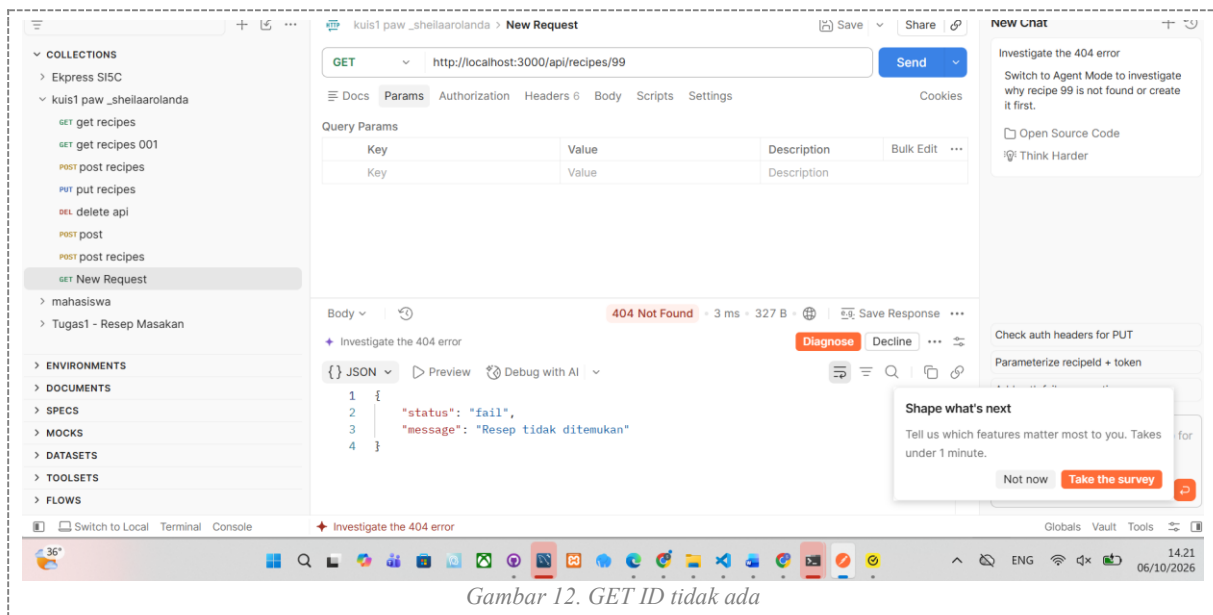
Penjelasan:

Pengujian ini menguji penanganan error sistem (*error handling middleware*) ketika menerima payload bertipe JSON yang salah format. permintaan POST dikirim dengan sintaksis JSON yang rusak (seperti koma berlebih). Middleware errorHandler berhasil menangkap kesalahan parser tersebut dan merespons dengan status 400 Bad Request berbentuk JSON.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (<i>request</i>)	GET /[sumber-hari]/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[] Ya [] Tidak



Gambar 12. GET ID tidak ada

Gambar 12. GET ID tidak ada

Penjelasan:

Pengeujian ini bertujuan untuk memastikan penanganan sistem ketika klien meminta data resep yang tidak terdaftar. permintaan dikirim dengan metode GET ke endpoint /api/recipes/99. Server memeriksa database/array data, tidak menemukan resep dengan ID tersebut, lalu mengembalikan respon 404 Not Found berisi pesan error JSON..

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	
T02	Menampilkan satu data berdasarkan ID	200	200	
T03	Menambah data baru	201	201	
T04	Mengubah data	200	200	
T05	Menghapus data	204	204	
N01	Menambah data tanpa API key	401	401	
N02	Menambah data dengan body tidak lengkap	400	400	
N03	Mengirim JSON yang rusak	400	400	
N04	Mengambil ID yang tidak ada	404	404	

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	
Pesan commit terakhir	
Folder routes, controllers, models, middlewares ada	☐ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☐ Ya ☐ Tidak
.env.example ada	☐ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

alam arsitektur MVC resep ini, Route berfungsi mengarahkan permintaan HTTP ke fungsi yang tepat (contoh: `router.get('/', recipeController.getAllRecipes)`), Controller memproses logika bisnis dan validasi data (contoh: `const newRecipe = recipeModel.createRecipe(req.body)`), sedangkan Model mengelola struktur serta manipulasi penyimpanan data resep (contoh: `recipes.push(newRecipe)`). Ketiganya saling memisahkan tanggung jawab kode agar aplikasi lebih terstruktur dan mudah dikembangkan.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Keuntungan utama *error handler* mogok adalah menghindari duplikasi kode serta memastikan format respons kesalahan (seperti JSON) konsisten di seluruh endpoint tanpa perlu ditulis ulang secara manual. Buktinya terlihat pada pengujian N03 dan N04, di mana error format JSON rusak maupun rute tidak ditemukan otomatis ditangani oleh middleware keinginan untuk menghasilkan respon 400 Bad Request atau 404 Not Found berformat JSON secara seragam.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [...] sudah diganti dengan topik Tugas 1 saya	
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	
4	Identitas saya (nama atau NIM) muncul pada data uji POST	
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	
6	Tabel rekapitulasi sesuai dengan tangkapan layar	
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggah	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100